

INTRODUCCIÓN A LA PROGRAMACIÓN EN SCL

ALEJANDRO RAMIRO OLMO Y JOSE MANUEL RODRÍGUEZ EXPÓSITO

ÍNDICE

Vision General

- ¿QUÉ ES SCL? >
- SCL EN PORTAL TIA >
- VARIABLES >
- EXPRESIONES Y OPERADORES LOGICOS >

FC y FB

- BLOQUES FC Y FB >

Empezando a programar

- BASES DE DATOS >
- CONDICIONALES >
- BUCLES >
- ORGANIZACIÓN >

Ejercicio

- DETECCIÓN DE FLANCO >

Contadores y temporizadores

- TEMPORIZADORES >
- CONTADORES >

Conclusiones

- VENTAJAS >
- CONCLUSIÓN FINAL >

Visión general de SCL

00



QUE ES SCL

STRUCTURED CONTROL LANGUAGE



- Uso para PLC
- Basado en Pascal
- Texto estructurado, a diferencia del lenguaje de contactos
- IEC 61131-3 (ST).

00

```
1 IF #req THEN
2
3     #temp_media := (#var1 + #var2 + #var3) / 3;
4
5 IF #temp_media > #Lim_Val THEN
6     #Error := true;
7     #result := -99.9;
8 ELSE
9     #result := #temp_media;
10    #Error := false;
11 END_IF;
12 (* comentario 1
13 comentario 2
14 comentario 3*)
15
16 CASE variable_name OF
17     1: // Statement section case 1
18         ;
19     2..4: // Statement section case 2 to 4
20         ;
21     ELSE // Statement section ELSE
22         ;
23 END_CASE;
24
25
26 END_IF;
```


SCL VS KOP



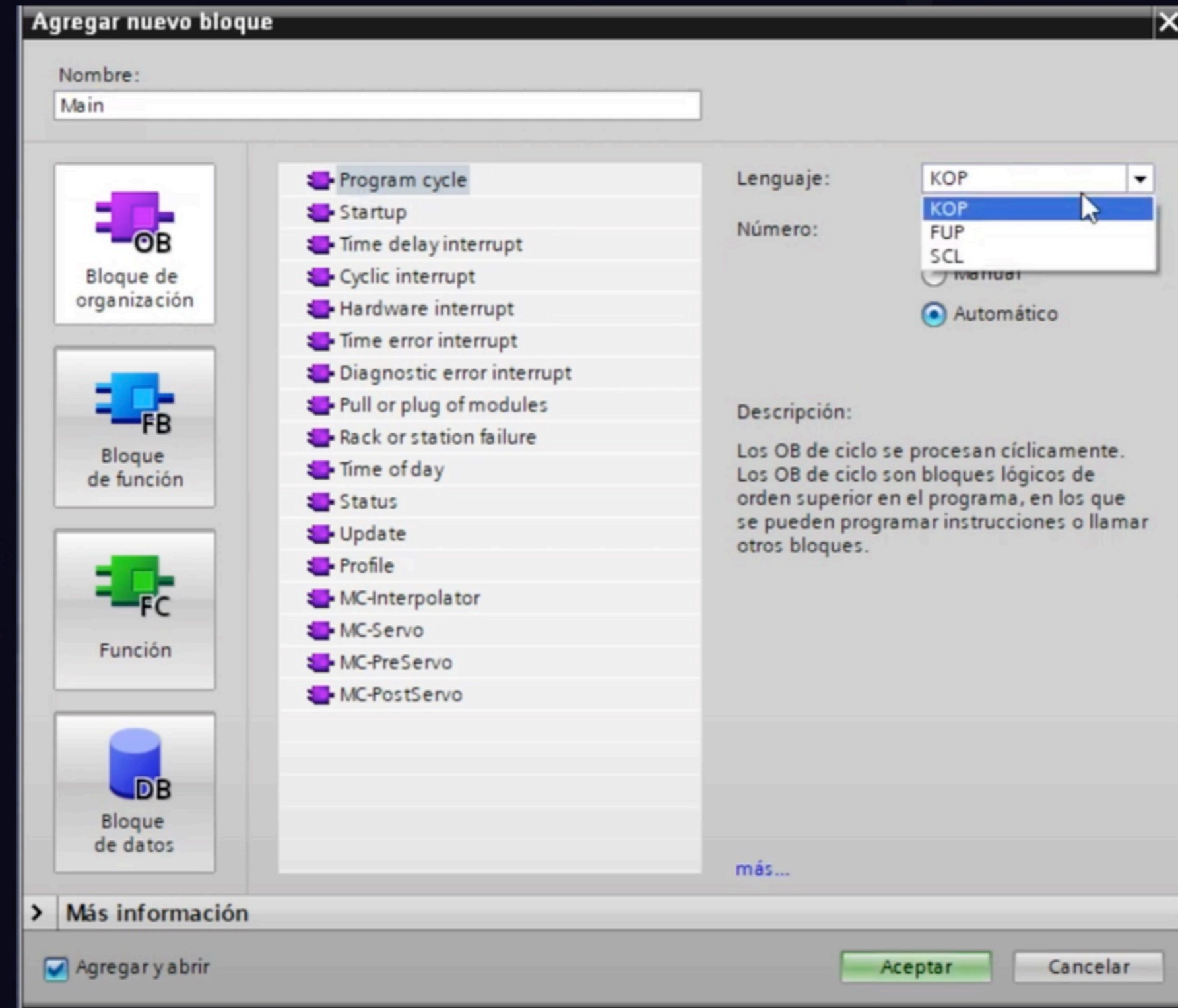
- SCL: Superior en potencia y estructuración. Ideal para proyectos grandes o donde se necesiten bases de datos con calculos complejos (arrays)
- KOP: Más sencillo, visual e intuitivo . Permite depurar facilmente

Importante diferenciar programación con SCL en TIA Portal y STEP7

	SCL	KOP
Visual	✗	✓
Manipulación de datos	✓	✗
Estructuración	✓	✗
Rapidez en la programación	✓	✗

EMPEZAR A PROGRAMAR EN PORTAL TIA

Abrir un bloque de programa en
SCL



00

Visión general de SCL

VARIABLES



- No varían de un lenguaje a otro.
- El uso de las variables en KOP está más limitado. En SCL más aplicabilidad.

Tipo de dato	Descripción
BOOL	1 bit (TRUE/FALSE)
INT , DINT	16/32 bits (negativos en A2)
REAL	Coma flotante (IEEE-754)
CHAR , STRING	Cadena de caracteres
TIME	Duración (T#5s)

EXPRESIONES

Junto con los operadores lógicos
permitirán establecer condiciones

Operador	Descripción
<	Menor que
>	Mayor que
<=	Menor o igual que
>=	Mayor o igual que
=	Igualdad
<>	Diferente

OPERADORES LÓGICOS

Con variables de tipo BOOL

Operador	Descripción
NOT	Negación lógica
AND (&)	Conjunción (Y)
OR	Disyunción (O)
XOR	Exclusivo (O exclusivo)

●●● Ejemplo AND

- A := TRUE;
- B := FALSE;
- C := A AND B;

C -> FALSE

●●● Ejemplo OR

- A := TRUE;
- B := FALSE;
- C := A AND B;

C -> TRUE

00

OPERADORES LÓGICOS

Con variables de tipo INT

00

```
00000101 (5)
AND 00000011 (3)
-----
00000001 (1)
```

```
00000101 (5)
OR  00000011 (3)
-----
00000111 (7)
```

Importante tener en cuenta el comportamiento de los operadores lógicos si utilizamos enteros. Puede llevar a errores en el programa.

```
00000101 (5)
XOR 00000011 (3)
-----
00000110 (6)
```


Empezando a programar

01

BASE DE DATOS

¿Qué utilidad tienen en SCL?



Para aprovechar al máximo SCL, estudiaremos brevemente las bases de datos

Agregar nuevo bloque

Nombre: DBG

 Bloque de organización

 Bloque de función

 Función

 Bloque de datos

Tipo: DB global

Lenguaje: DB

Número: 2

☐ Manual

☒ Automático

Descripción:
Los bloques de datos (DB) sirven para almacenar datos del programa.

[más...](#)

> Más información

☒ Agregar y abrir

Aceptar Cancelar

01

Empezando a programar

INS

Instancias

Las instancias solo pueden ser llamadas por el FB (Contadores y Temporizadores son considerados FB's y van asociados a instancias)

DBG

Bloque de datos global

Pueden ser llamadas en cualquier momento del programa por cualquier tipo de bloque.

BASE DE DATOS

Tipos de datos en un DB General

Tipo de dato	Descripción
BOOL	1 bit (TRUE/FALSE
INT , DINT	16/32 bits (negativos en A2)
REAL	Coma flotante (IEEE-754)
CHAR , STRING	Cadena de caracteres
TIME	Duración (T#5s)

Tipo	Categoría	Descripción
ARRAY	Estructura	Lista de elementos del mismo tipo
STRUCT	Estructura	Agrupación de distintos tipos de datos

Empezando a programar

BASE DE DATOS

Estructura final de un DB General

01

GDB ▶ PLC [CPU 1211C DC/DC/DC] ▶ Bloques de programa ▶ DBG [DB2]

Conservar valores actuales Instantánea Copiar instantáneas a valores de arranque Cargar valores de arranque como valores actuales

DBG

	Nombre	Tipo de datos	Valor de arranq...	Remanencia	Accesible d...	Escrib...	Visible en ..	Valor de a..	Comentario
1	▼ Static			☐	☐	☐	☐	☐	
2	▼ MODO	Struct		☐	☒	☒	☒	☐	
3	Instalacion_en servicio	Bool	false	☐	☒	☒	☒	☐	Instalacion en automatico
4	▼ COLORES	Struct		☐	☒	☒	☒	☐	
5	Pieza azul	Bool	false	☐	☒	☒	☒	☐	reconoce pieza azul
6	Pieza verde	Bool	false	☐	☒	☒	☒	☐	reconoce pieza verde
7	Pieza gris	Bool	false	☐	☒	☒	☒	☐	reconoce pieza gris
8	▼ DATOS	Struct		☒	☒	☒	☒	☐	
9	cantidad de piezas en entero	Int	0	☒	☒	☒	☒	☐	piezas contadas en un valor entero
10	Cantidad de piezas en Real	Real	0.0	☒	☒	☒	☒	☐	piezas contadas en real
11	tiempo de funcionamiento de la instal.	DWord	16#0	☒	☒	☒	☒	☐	tiempo de funcionamiento de la instalacion e..
12	piezas por segundo	Real	0.0	☒	☒	☒	☒	☐	resultado piezas por segundo
13	▼ OTROS	Struct		☐	☒	☒	☒	☐	
14	marca FP	Bool	false	☐	☒	☒	☒	☐	marca de ayuda flanco positivo

Empezando a programar

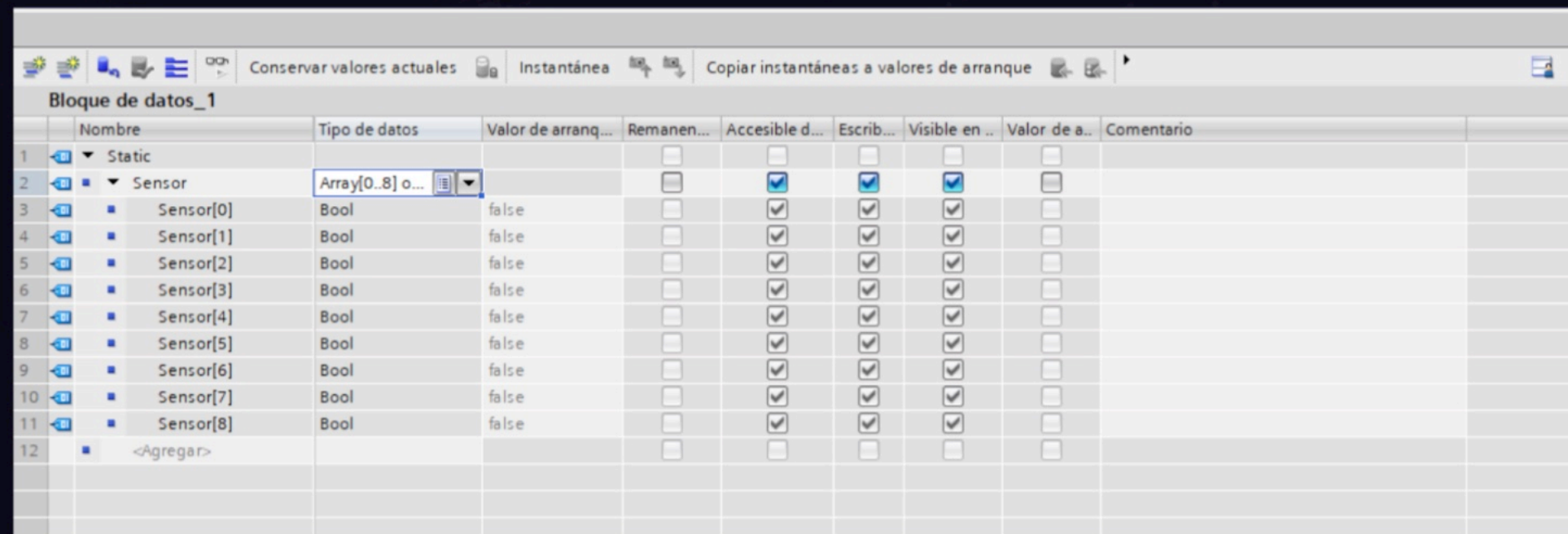
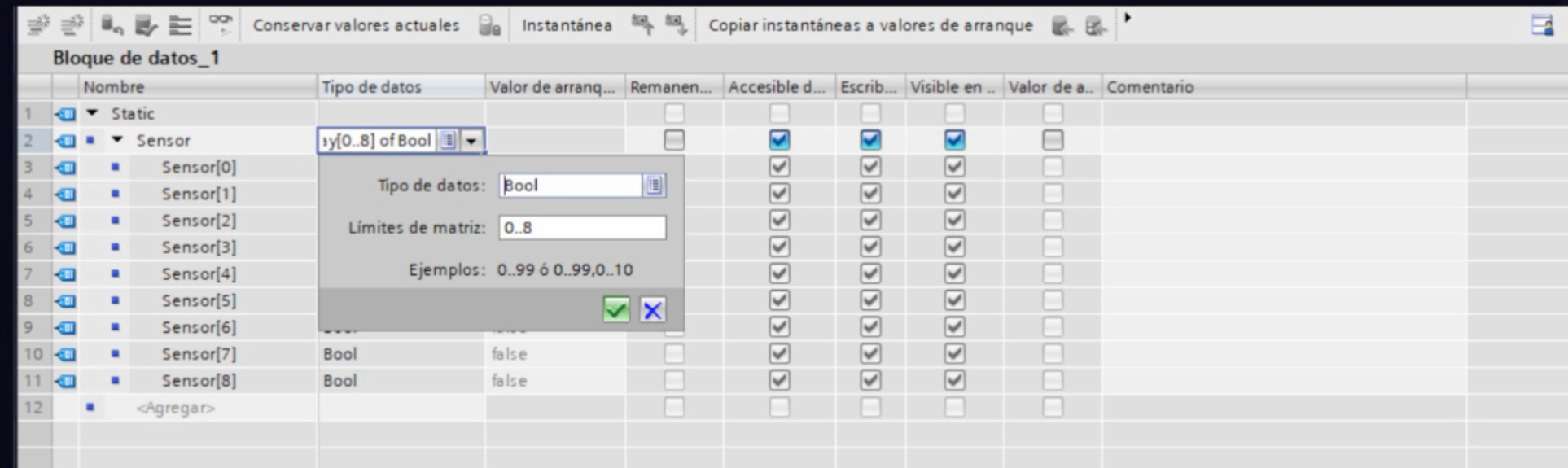
BASES DE DATOS

Trabajar con Arrays



Para trabajar con arrays es necesario crear una base de datos.

Dentro de la base de datos debemos crear una variable de tipo "Array" y definimos los límites.



Empezando a programar

INTSTRUCCIONES PRINCIPALES

Organización

Condicional

IF / CASE



Bucles

FOR / WHILE

Empezando a programar

IF

Ejecuta un bloque de código si se cumple la condición indicada

Los comandos principales son IF, ELSIF y ENDIF

CASE

Ejecuta un bloque de código u otro en función de una variable de tipo INT

EJEMPLO ESTRUCTURAS CONDICIONALES

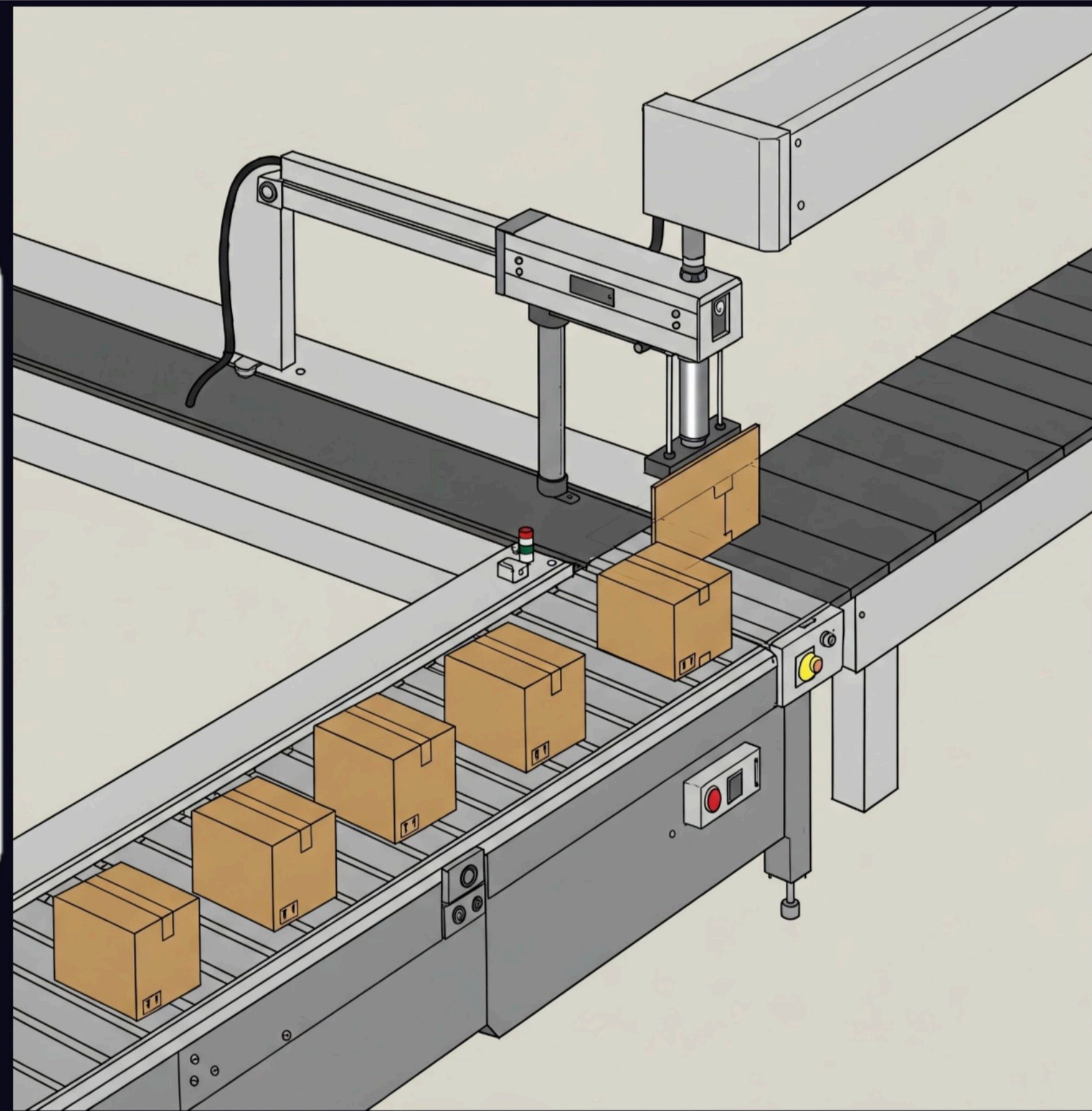


Disponemos de una cinta transportadora (B) la cual transporta cajas de cartón vacías. El cilindro de doble efecto (A) las retira.

El sensor de peso (Sp) detectará la presencia de la caja

La instalación se pondrá en marcha con un pulsador (Start) y se detendrá con otro pulsador (Stop).

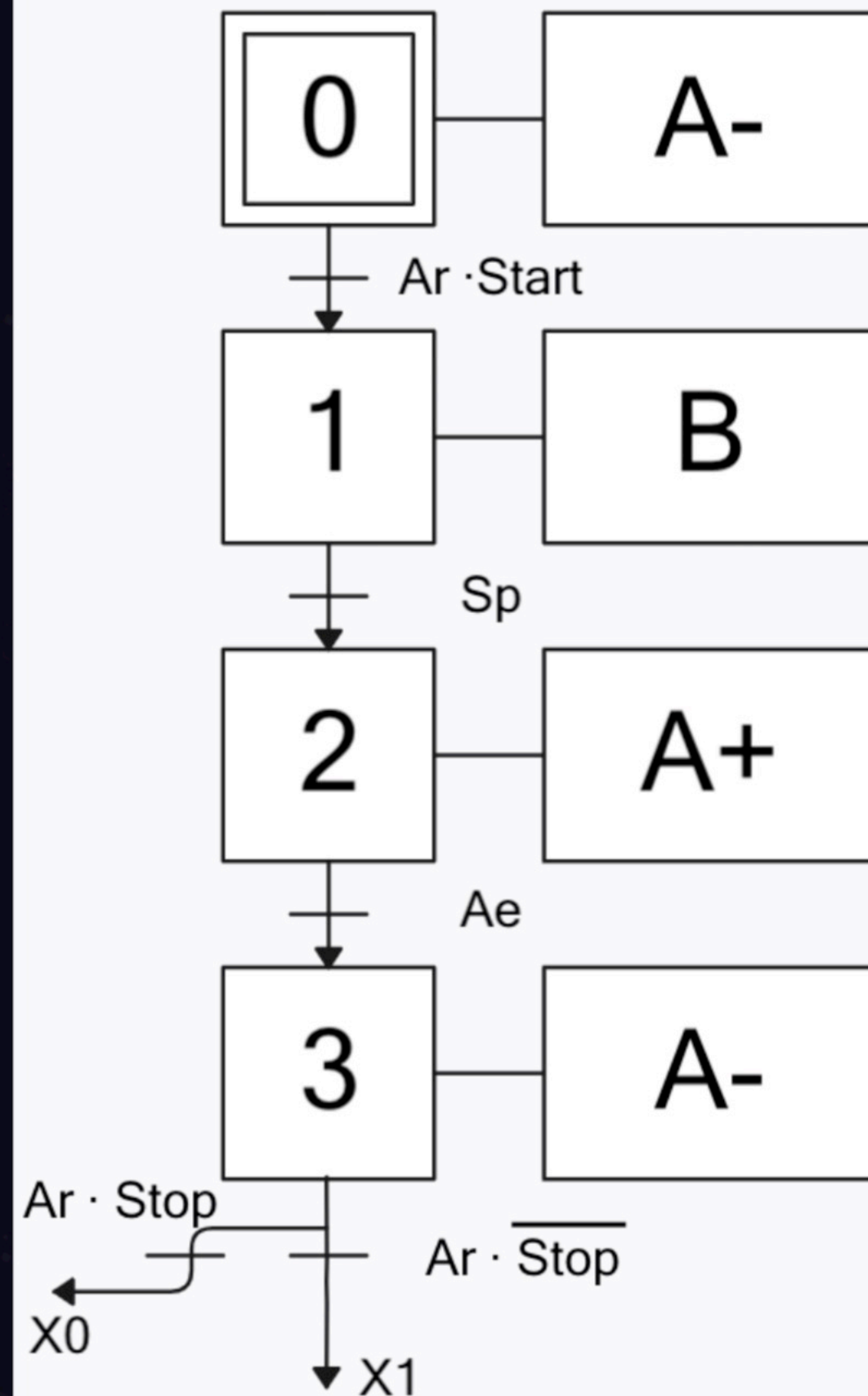
Considerar que el pulsador de start detecta flanco positivo por si mismo.



01

Empezando a programar

EJEMPLO ESTRUCTURAS CONDICIONALES



01

Empezando a programar

EJEMPLO ESTRUCTURAS CONDICIONALES

Resolución con IF

```
IF... CASE... FOR... WHILE... (*...*) REGION
OF... TO DO... DO...

1
2 // Máquina de estados usando IF en lugar de CASE
3
4 // Estado 0: Espera de inicio
5 IF "Estado" = 0 THEN
6     IF "start" THEN
7         "Estado" := 1;
8     END_IF;
9 END_IF;
10
11 // Estado 1: Mover cinta hasta sensor Sp
12 IF "Estado" = 1 THEN
13     "B+" := TRUE;
14     IF "Sp" THEN
15         "Estado" := 2;
16     END_IF;
17 END_IF;
18
19 // Estado 2: Avanzar cilindro (A+)
20 IF "Estado" = 2 THEN
21     "A-" := FALSE;
22     "A+" := TRUE;
23     IF "A+" THEN
24         "Estado" := 3;
25     END_IF;
26 END_IF;
27
28 // Estado 3: Retroceder cilindro (A-)
29 IF "Estado" = 3 THEN
30     "A+" := FALSE;
31     "A-" := TRUE;
32
33     IF "Ar" AND "stop" THEN
34         "Estado" := 0; // Parar todo
35     ELSIF "Ar" AND NOT "stop" THEN
36         "Estado" := 1; // Repetir ciclo automáticamente
37     END_IF;
38 END_IF;
39
```

Empezando a programar

EJEMPLO ESTRUCTURAS CONDICIONALES

Resolución con CASE

```
IF... CASE... FOR... WHILE... (*...*) REGION
OF... TO DO... DO...

1 // Máquina de estados
2 CASE "Estado" OF
3
4     0: // Espera de inicio
5
6     IF "start" THEN
7         "Estado" := 1;
8     END_IF;
9
10    1: // Mover cinta hasta sensor Sp
11        "B+" := True;
12    IF "Sp" THEN
13        "Estado" := 2;
14    END_IF;
15
16    2: // Avanzar cilindro (A+)
17        "A-" :=false;
18        "A+" :=TRUE;
19    IF Ae THEN
20        "Estado" := 3;
21    END_IF;
22
23    3: // Retroceder cilindro (A-)
24        "A+":=FALSE;
25        "A-":=TRUE;
26    IF "Ar" AND "stop" THEN
27        "Estado" := 0; // Parar todo
28    ELSIF "Ar" AND NOT "stop" THEN
29        "Estado" := 1; // Repetir ciclo automáticamente
30    END_IF;
31
32 END_CASE;
33
```


FOR

Ejecuta un bloque de código un número determinado de veces.

Funciona con una variable de control que se incrementa o decrementa en cada ciclo.

WHILE

Ejecuta un bloque de código mientras que la condición sea verdadera

El numero de repeticiones del bucle puede no ser siempre el mismo.

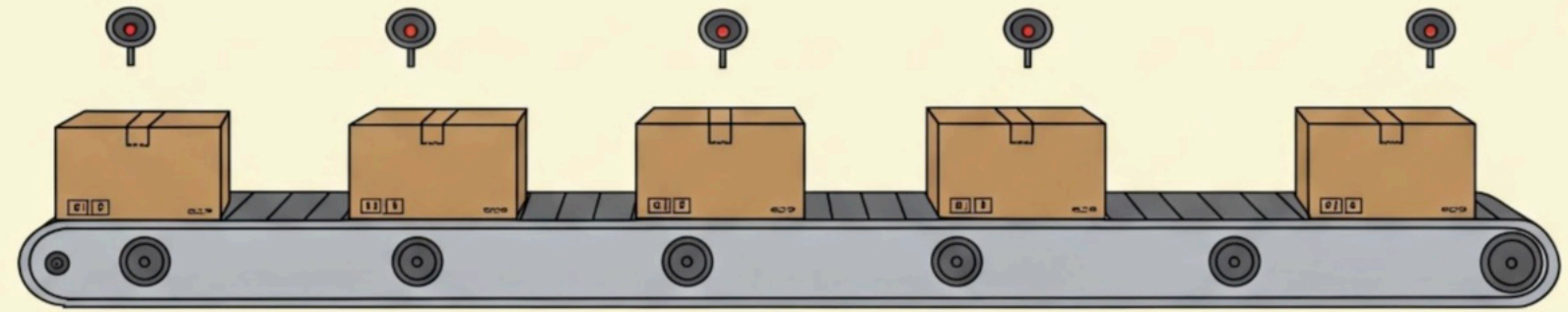
*Importante: Estas instrucciones realizan la totalidad de ciclos en un solo ciclo SCAN

EJEMPLO BUCLES



En una planta de control de calidad disponemos de un sistema con 8 cámaras, cada una encargada de revisar una parte del producto, las cuales devuelven el resultado en un array de longitud 8.

Si no hay ninguna anomalía el sensor dará un valor booleano TRUE, de lo contrario dará FALSE.



EJEMPLO BUCLES

Resolución con FOR

```
1 // Lógica principal
2 IF "start" THEN
3     // Asumimos que el producto es bueno, hasta que se demuestre lo contrario
4     "Producto_OK" := TRUE;
5
6     // Usamos FOR para revisar TODOS los sensores
7     FOR #i := 1 TO 8 DO
8         IF NOT "Bloque de datos_1".Sensor[#i] THEN
9             // Si uno falla, el producto es malo
10            "Producto_OK" := FALSE;
11            "Producto_Malo" := TRUE;
12            EXIT; // Salimos del bucle, no hace falta revisar más
13        END_IF;
14    END_FOR;
15
16    // Si después del FOR no se marcó como malo → es bueno
17    IF "Producto_OK" THEN
18        "Producto_Malo" := FALSE;
19    END_IF;
20 END_IF;
21
22
```


EJEMPLO BUCLES

Resolución con WHILE

```
IF "start" THEN
    // Asumimos que el producto es bueno, hasta que se demuestre lo contrario
    "Producto_OK" := TRUE;

    // WHILE para revisar TODOS los sensores
    #i := 1;
    WHILE (#i <= 8) AND "Producto_OK" DO
        IF NOT "Bloque de datos_1".Sensor[#i] THEN
            // Si uno falla, el producto es malo
            "Producto_OK" := FALSE;
            "Producto_Malo" := TRUE;
            // Salimos del ciclo de forma natural porque Producto_OK es FALSE
        END_IF;
        #i := #i + 1;
    END_WHILE;

    // Si después del WHILE no se marcó como malo → es bueno
    IF "Producto_OK" THEN
        "Producto_Malo" := FALSE;
    END_IF;
END_IF;
```

BLOQUES DE ORGANIZACIÓN

(*...*)

No tiene repercusión en el código.
Su función es permitir
comentarios multilinea.

REGION

Permite organizar el código en
regiones, siendo posible asignarle
a cada una un nombre

```
IF... CASE... FOR... WHILE... (*...*) REGION
OF... TO DO... DO...

14 Region Activación de los flancos para el contador del parking
15 Region Flanco "Descendente" para el conteaje ascendente de vehiculos en el parking
16 "F_TRIG_INCREMENTO"(CLK:=Sensor Entrada",
17 Q=>"var"."Cont/Temp".Contadores."Contaje Ascendente");
18 End_Region
19 Region Flanco "Descendente" para el conteaje descendente de vehiculos en el parking
20 "F_TRIG_DECREMENTO"(CLK:=Sensor Salida",
21 Q=>"var"."Cont/Temp".Contadores."Contaje Descendente");
22 End_Region
23 End_Region
24
25 Region Contador de vehiculos para el parking
26 "var"."Cont/Temp".Contadores.CTUD.CTUD(CU:="var"."Cont/Temp".Contadores."Contaje Ascendente" And Not "var".General.Sistemas."Parking Completo",
27 CD:="var"."Cont/Temp".Contadores."Contaje Descendente" And Not "var".General.Sistemas.Inicial,
28 R:="var"."Cont/Temp".Contadores.ResetCount,
29 PV:="var"."Cont/Temp".Contadores.CTUD.PV,
30 CV=>"var"."Cont/Temp".Contadores.CTUD.CV);
31 End_Region
32
33 Region Temporizadores
34 Region Temporizador para la bajada de la barrera de entrada
35 "var"."Cont/Temp".Temporizadores.TemporizadorEntrada.TON(IN := "var"."Cont/Temp".Temporizadores."Act.TempEntrada",
36 FT := T#10s,
37 Q => "var"."Cont/Temp".Temporizadores.TempEntradaOk,
38 ET => "var"."Cont/Temp".Temporizadores.TempTimeEntrada);
39 End_Region
40 Region Temporizador para la bajada de la barrera de salida
41 "var"."Cont/Temp".Temporizadores.TemporizadorSalida.TON(IN:= "var"."Cont/Temp".Temporizadores."Act.TempSalida",
42 FT:=T#5s,
43 Q=>"var"."Cont/Temp".Temporizadores.TempSalidaOk,
44 ET=>"var"."Cont/Temp".Temporizadores.TempTimeSalida);
45 End_Region
46 Region Temporizador de impulso mediante flanco
47 "var"."Cont/Temp".Temporizadores.TemporizadorImpulso.TP(IN := "var"."Cont/Temp".Contadores."Contaje Ascendente",
48 FT := T#5s,
49 Q => "SalidaTempImpulso",
50 ET => "var"."Cont/Temp".Temporizadores.TempTimeImpulso);
51 End_Region
52 End_Region
53 Region Asignación del valor PV al conteaje
54 "var"."Cont/Temp".Contadores.CTUD.PV := 10;
55 End_Region
56 End_Region
```


TEMPORIZADORES

Implementación de Temporizadores en SCL

02

TP**Pulso****TON****Retardo a
conexión****TOF****Retardo a
desconexión****TONR****Acumulativo**

Para implementarlos en TIA PORTAL:

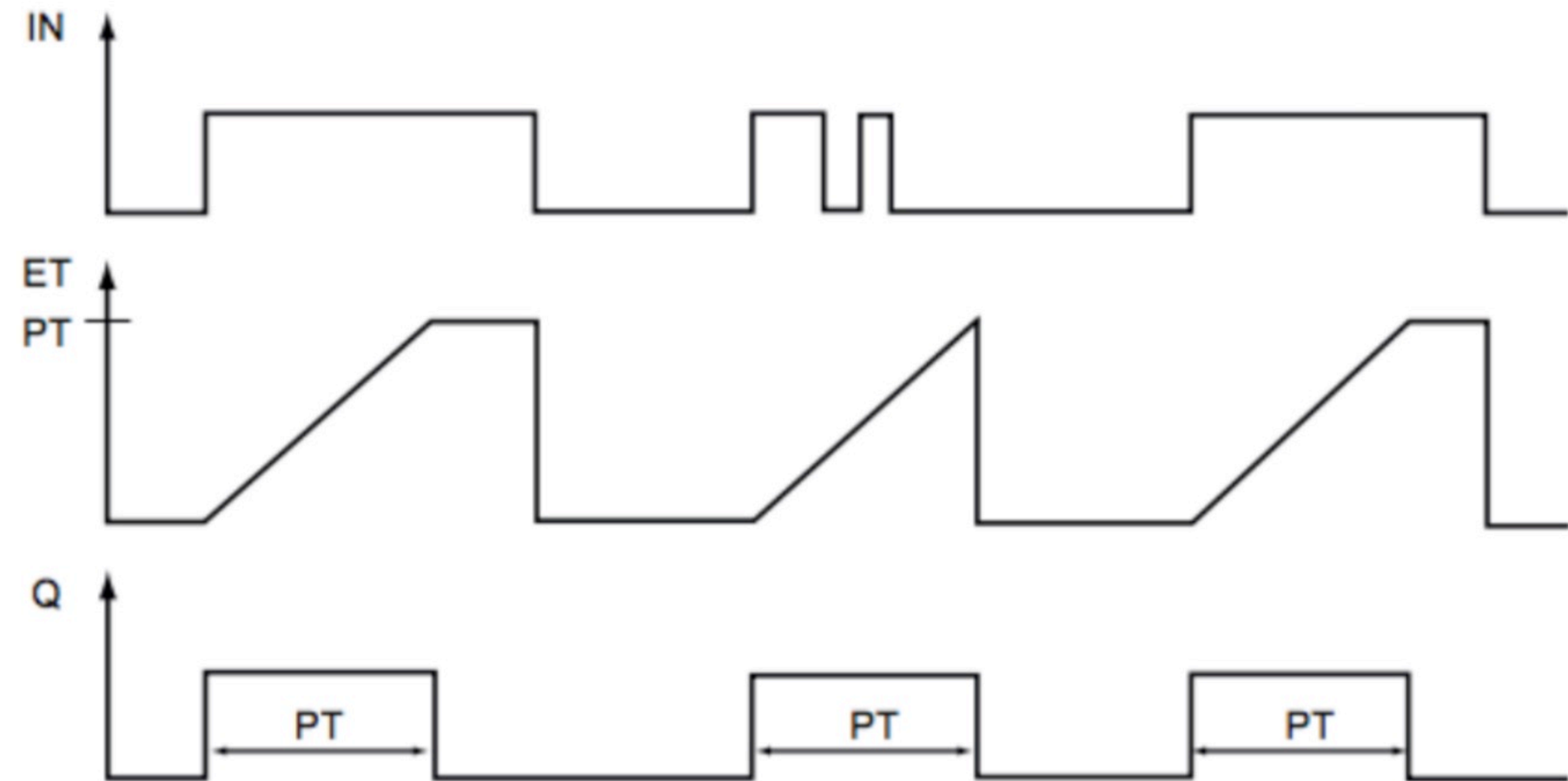
**Temporizadores**

TP

TEMPORIZADOR DE PULSO



- IN: Entrada de activación del temporizador
- PT: Tiempo del temporizador
- Q: Salida del temporizador
- ET: Tiempo transcurrido



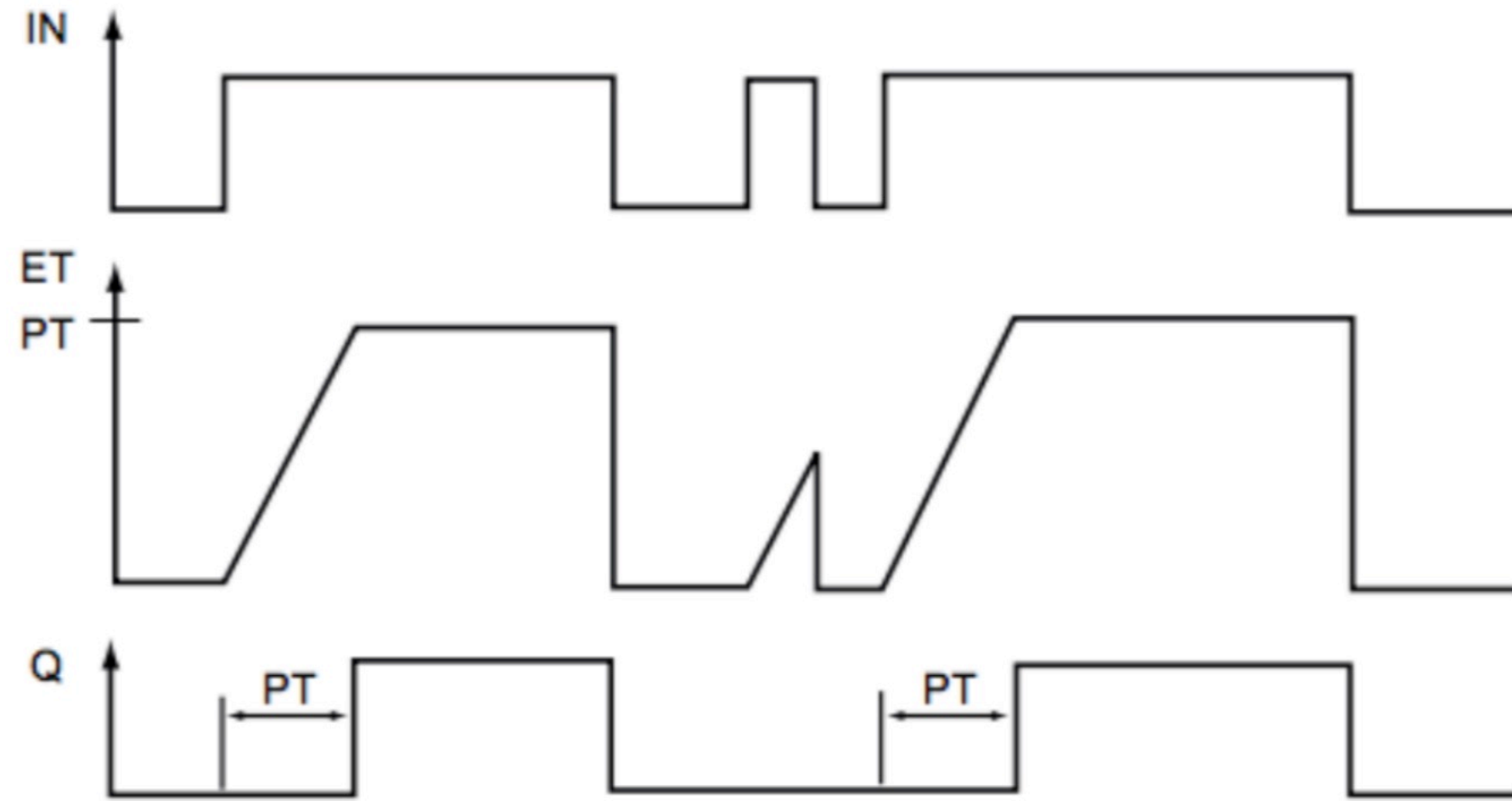
```
"IEC_Timer_0_DB_8".TP(IN:=_bool_in_,  
PT:=_time_in_,  
Q=>_bool_out_,  
ET=>_time_out_);
```

TON

TEMPORIZADOR RETARDO A LA CONEXIÓN



- IN: Entrada de activación del temporizador
- PT: Tiempo del temporizador
- Q: Salida del temporizador
- ET: Tiempo transcurrido



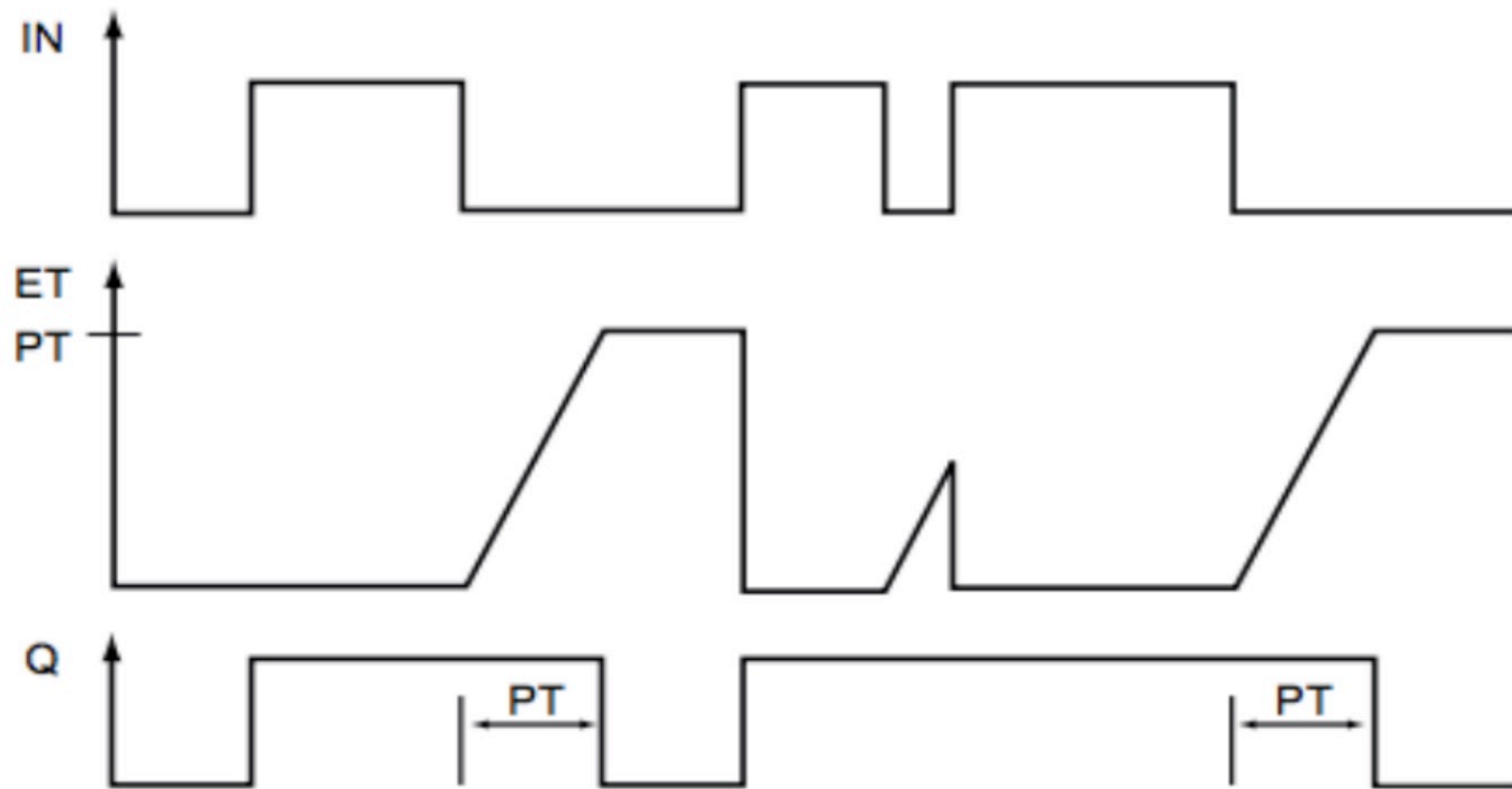
```
"Temporizador de la bomba 1".TON(IN:=bool_in,  
PT:=time_in,  
Q=>bool_out,  
ET=>time_out);
```


TOF

TEMPORIZADOR RETARDO A LA DESCONEXIÓN



- IN: Entrada de activación del temporizador
- PT: Tiempo del temporizador
- Q: Salida del temporizador
- ET: Tiempo transcurrido



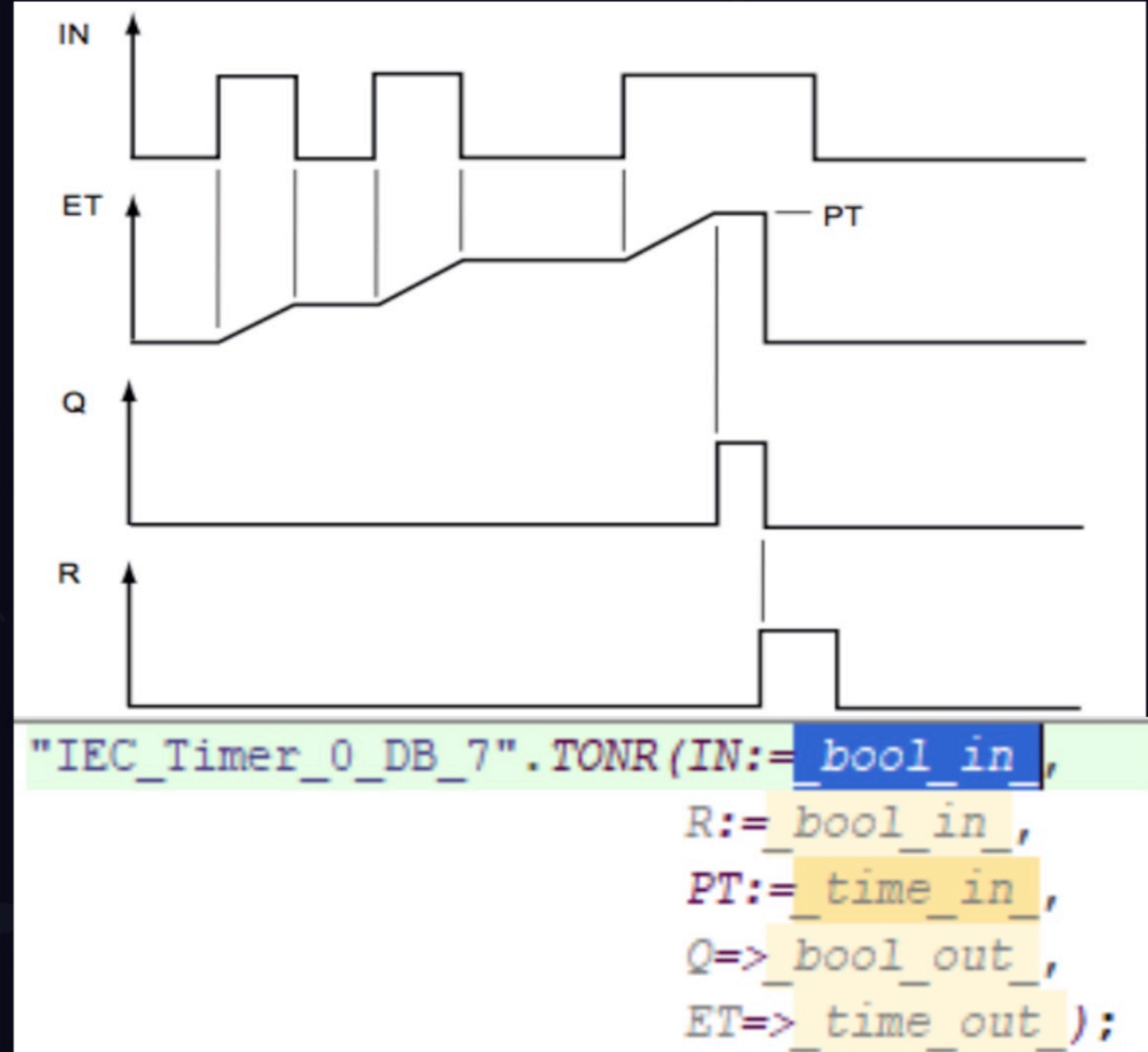
```
"IEC_Timer_0_DB_5".TOF(IN:=_bool_in_,  
                        PT:=_time_in_,  
                        Q=>_bool_out_,  
                        ET=>_time_out_);
```

TONR

TEMPORIZADOR ACUMULATIVO



- IN: Entrada de activación del temporizador
- R: Resetea el temporizador
- PT: Tiempo del temporizador
- Q: Salida del temporizador
- ET: Tiempo transcurrido



EJEMPLO CON TEMPORIZADOR

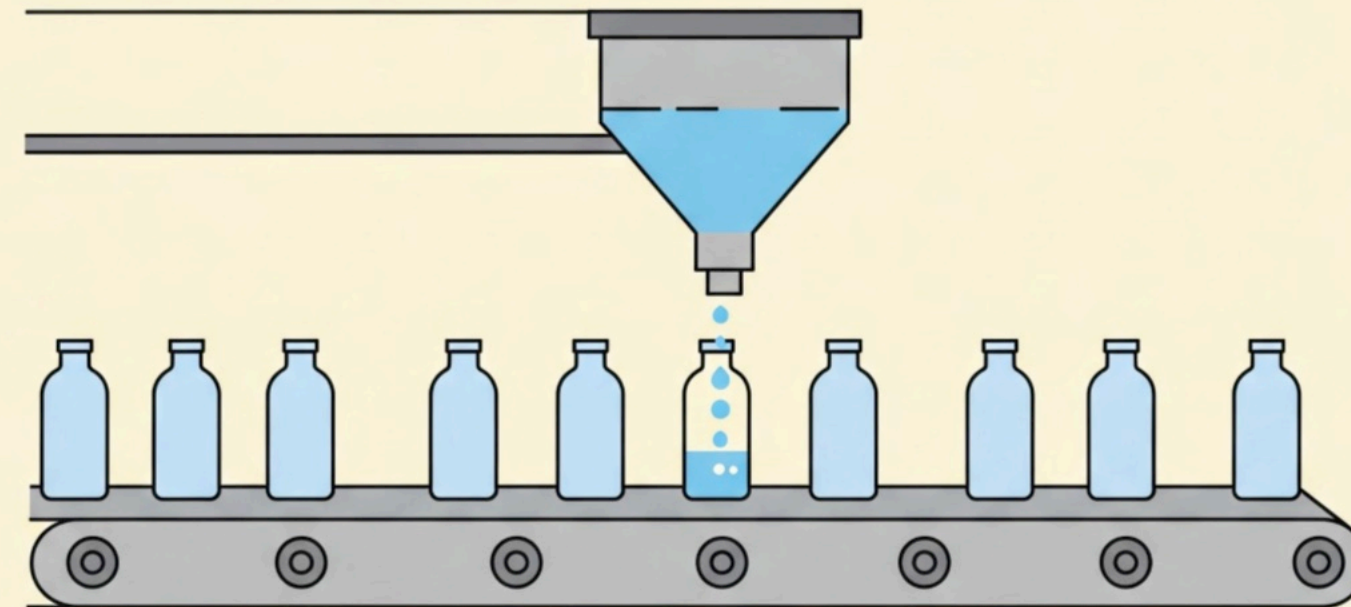
TEMPORIZADOR CON RETARDO A LA DESCONEXIÓN



Se desea automatizar el proceso de una planta embotelladora. Se dispone de una tolva con dos sensores, un sensor Sb indica que el nivel de líquido es insuficiente, activando una alarma para que un operario solucione la incidencia de manera manual.

El cilindro A de simple efecto se encarga del llenado, el cual debe durar 3 segundos

La cinta transportadora B se encarga del paso de los productos



EJEMPLO CON TEMPORIZADOR

TEMPORIZADOR CON RETARDO A
LA DESCONEXIÓN

02

```
IF... CASE... FOR... WHILE... (*...*) REGION
OF... TO DO... DO...

1 // Manejo de alarma
2
3 IF "SB" THEN
4     "alarm" := TRUE;
5     "A+" := FALSE;
6     // Resetear ambos temporizadores
7
8     "Estado" := 1000; // bloqueo
9
10 IF "Reset" AND "alarm" THEN
11     "alarm" := FALSE;
12     "Estado" := 0;
13 END_IF;
14 END_IF;
15
16 // Máquina de estados
17
18 CASE "Estado" OF
19
20     0: // Inicio ciclo - activar válvula
21         "A+" := TRUE;
22         "IEC_Timer_0_DB_4".TON(IN:="A+",
23                                 PT:=T#3s,
24                                 Q=>"Q1");
25
26 IF "Q1" THEN
27
28     "Estado" := 1;
29 END_IF;
30
31     1: // Pausa - válvula cerrada
32         "A+" := FALSE;
33         "IEC_Timer_0_DB_4".TON(IN := NOT "A+",
34                                 PT := T#6s,
35                                 Q => "Q2");
36
37 IF "Q2" THEN
38     "Estado" := 0; // volver a encender A+
39 END_IF;
40 END_CASE;
41
```

Temporizadores

CONTADORES

Implementación de Contadores en SCL

03

Contadores

CTU**ASCENDENTE**

Aumenta en uno el valor del contador cuando recibe un pulso ascendente

CTD**DESCENDENTE**

Decrementa el contador en uno cuando recibe un pulso

CTUD**ASCENDENTE/
DESCENDENTE**

Puede aumentar o decrementar el valor dependiendo de donde reciba el pulso

Para implementarlos en TIA PORTAL:

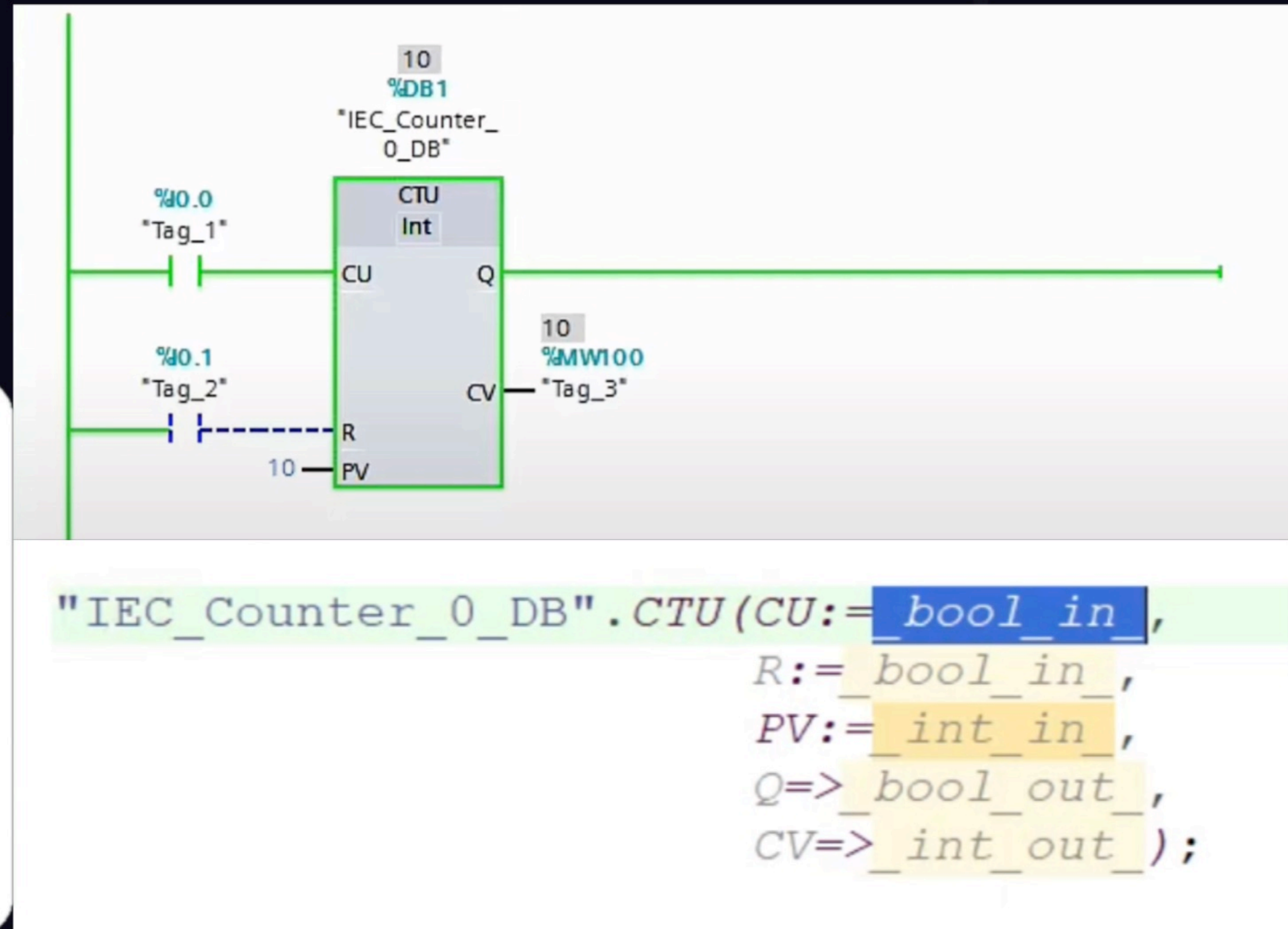


CTU

Contador Ascendente



- CU: cuando recibe un pulso ascendente incrementa el contador en 1
- R: Cuando recibe un pulso resetea el contador
- PV: Hasta donde queremos que cuente el contador
- Q: Da 1 cuando ha alcanzado el valor indicado en PV
- CV: Da el valor entero en el que está el contador

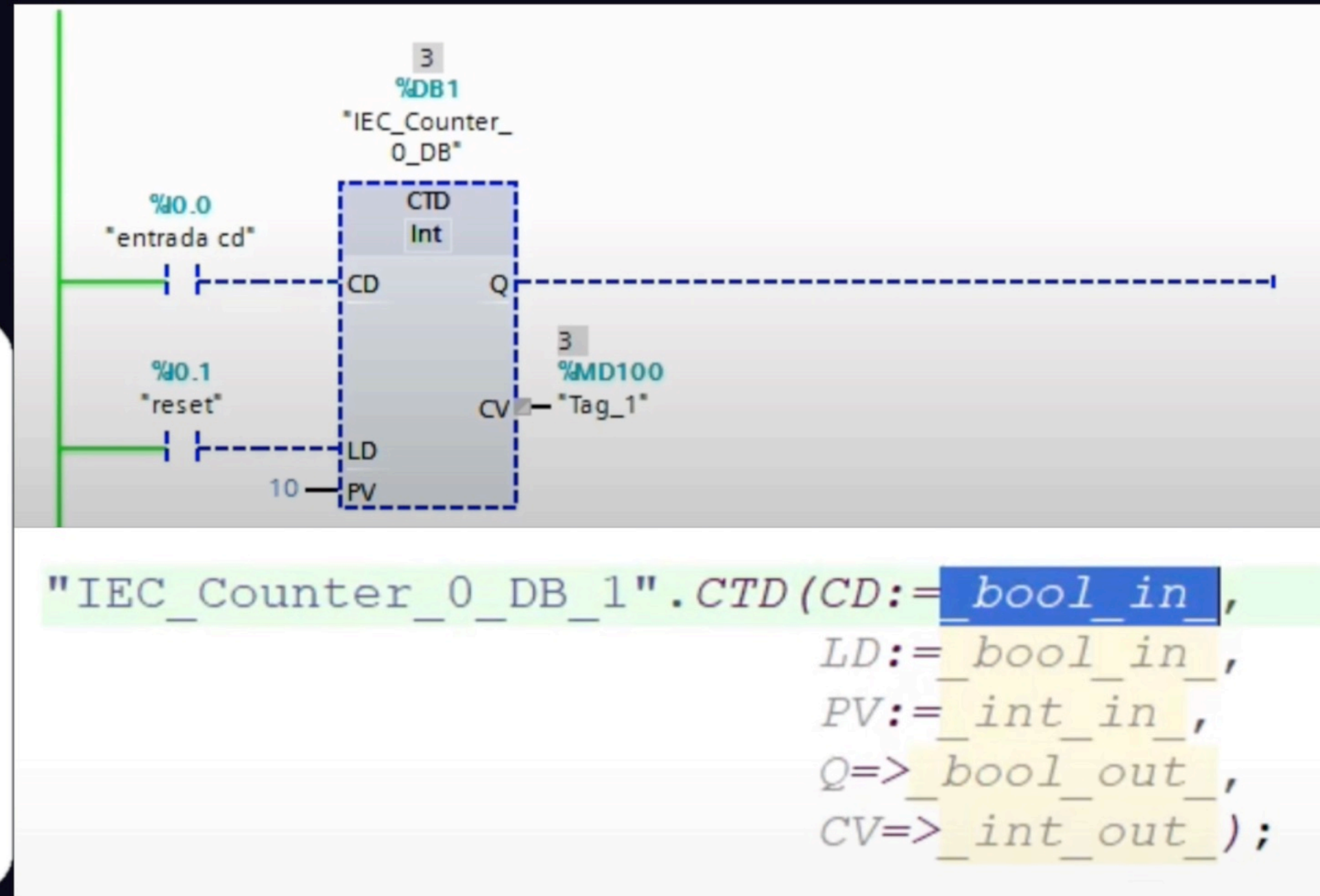


CTD

Contador Descendente



- CD: cuando recibe un pulso ascendente incrementa el contador en 1
- LD: Cuando recibe un pulso resetea el contador
- PV: Hasta donde queremos que cuente el contador
- Q: Da 1 cuando ha alcanzado el valor indicado en PV
- CV: Da el valor entero en el que está el contador

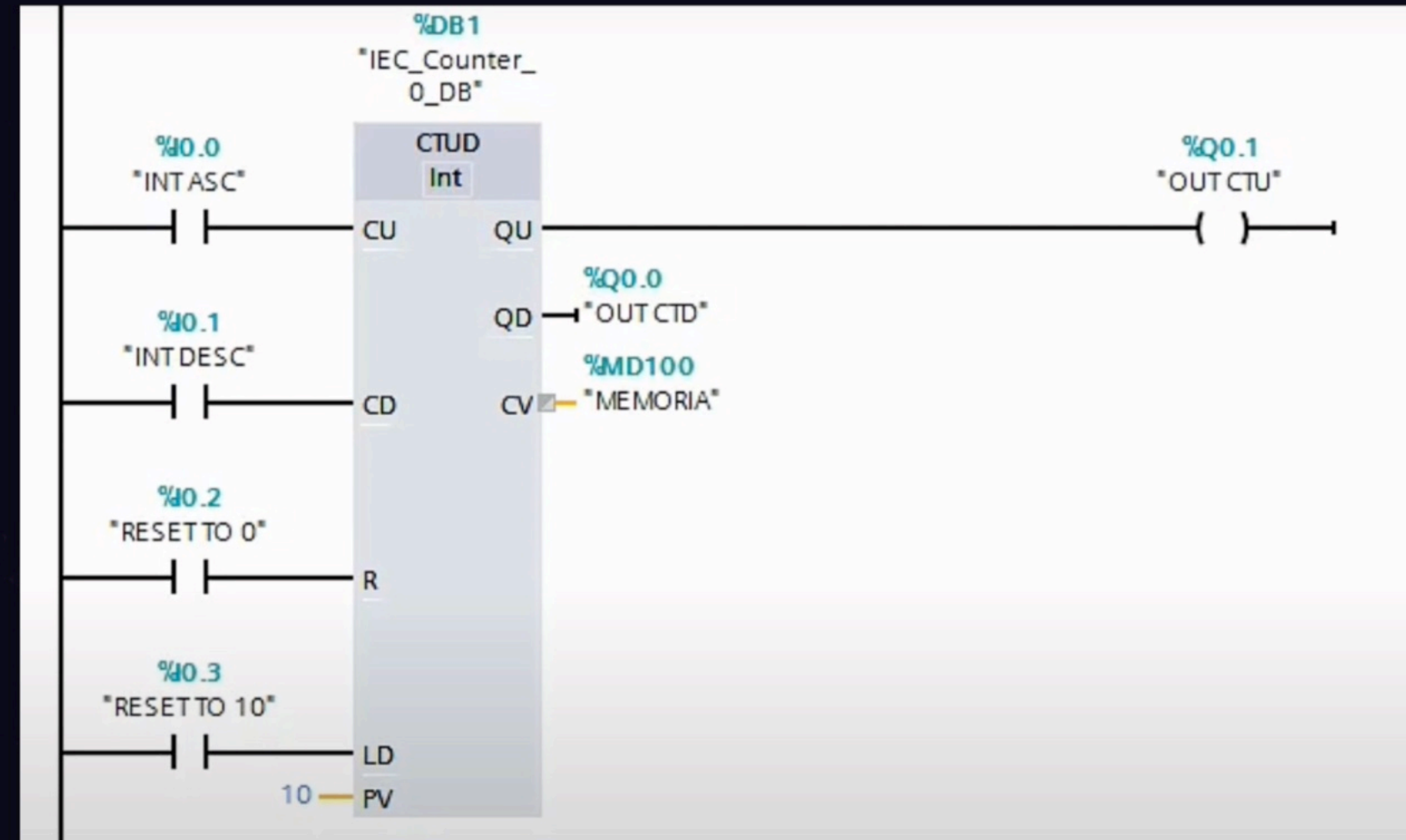


CTUD

Contador Ascendente/Descendente



- CU: cuando recibe un pulso ascendente incrementa el contador en 1
- CD: cuando recibe un pulso ascendente decrementa el contador en 1
- R: Resetea el contador al valor inicial
- LD: Resetea el contador al valor superior
- PV: Valor inicial o final del contador
- QU: Da un pulso cuando ha alcanzado el valor indicado en PV
- QD: Da un pulso cuando el contador llega a 0
- CV: Da el valor entero en el que está el contador



```
"IEC Counter 0 DB 1".CTUD(CU:=_bool_in_,  

CD:=_bool_in_,  

R:=_bool_in_,  

LD:=_bool_in_,  

PV:=_int_in_,  

QU=>_bool_out_,  

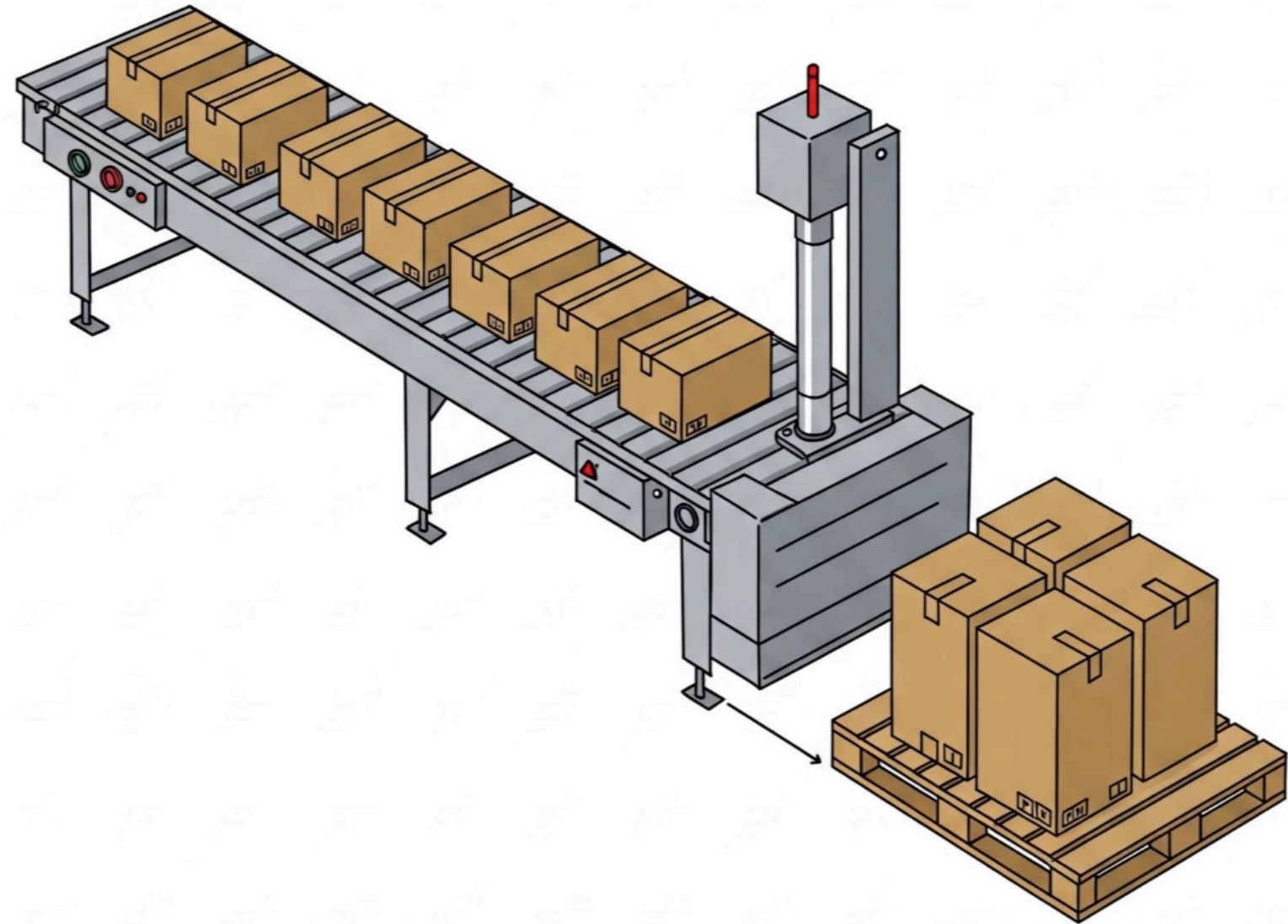
QD=>_bool_out_,  

CV=>_int_out_);
```

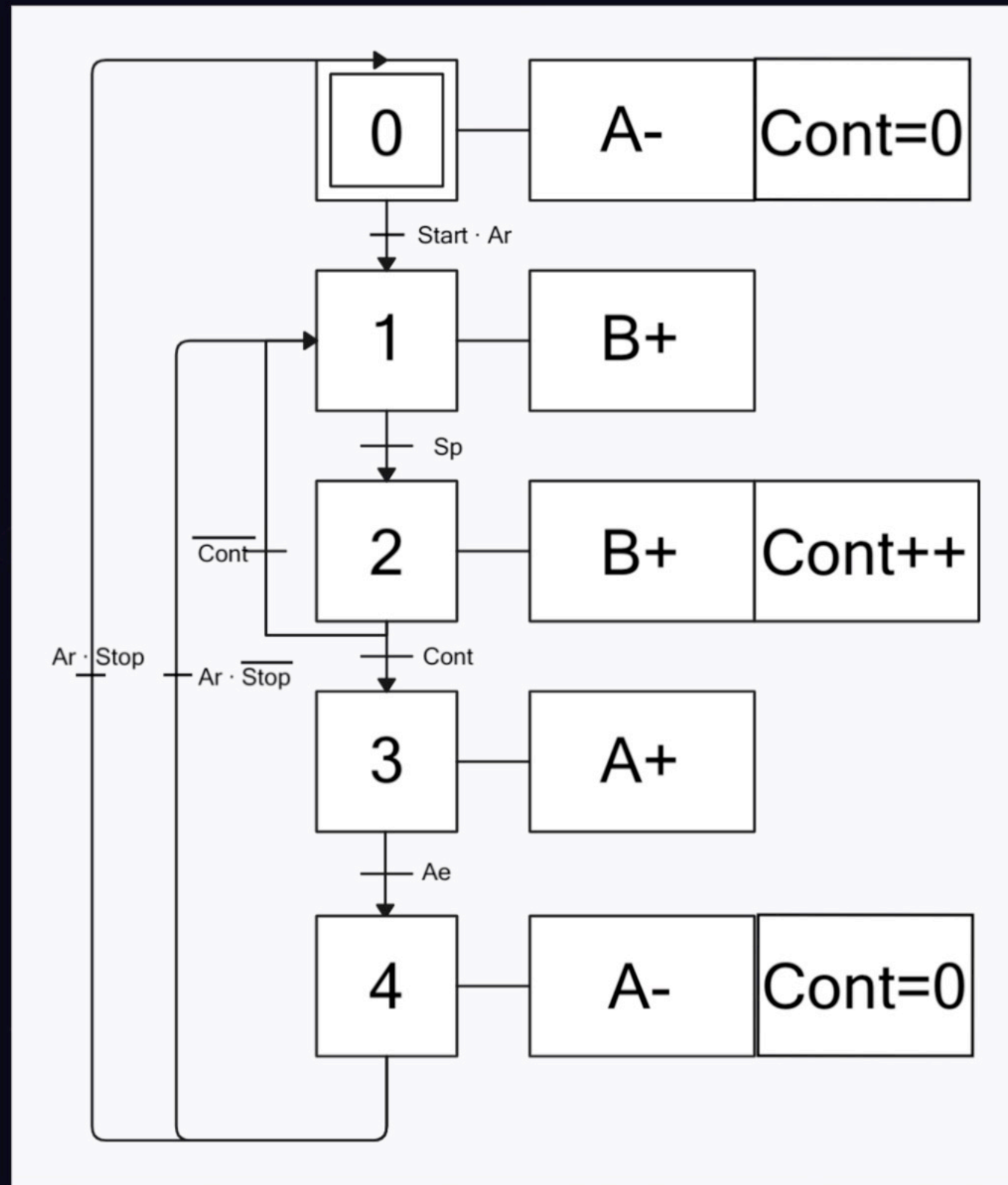

EJEMPLO CON CONTADORES



En una planta de producción en serie tenemos una cinta transportadora (B) la cual dispensa lotes de 10 productos a un cajón. Un sensor de presencia detecta el paso de los productos. El cilindro A de doble efecto retira el lote una vez estén los 10 productos.



EJEMPLO CON CONTADORES



EJEMPLO CON CONTADORES

```
1 //Ejemplo de "Contador"
2 "IEC_Counter_0_DB".CTU(CU:="Sp",
3     R:="Contador.R",
4     PV:=10,
5     Q=>"Contador.Q");
6 CASE "Estado" OF
7     0:
8         "A-":=TRUE;
9         "A+":= FALSE;
10        IF "start" AND "Ar" THEN
11            "Estado" := 1;
12        END_IF;
13
14
15    1: // Conteo de cajas
16        "A-" := FALSE;
17        "B+" := true;
18        IF "Contador.Q" THEN
19            "Estado" := 2;
20        END_IF;
21
22    2: // Activar cilindro A+
23        "A+":=TRUE;
24        "B+":=False;
25        IF "Ae" THEN // Esperar hasta que el cilindro llegue al final
26            "Contador.R" := TRUE; // Resetear el contador
27            "Estado" := 3;
28        END_IF;
29
30    3: // Esperar a que Ae se desactive (para evitar doble detección)
31        IF "Ar" AND "stop" THEN
32            "Contador.R" := FALSE;
33            "Estado" := 0; // Volver a contar
34        END_IF;
35        IF "Ar" AND NOT "stop" THEN
36            "Contador.R" := FALSE;
37            "Estado" := 1; // Volver a contar
38        END_IF;
39    END_CASE;
```


BLOQUES FC Y FB

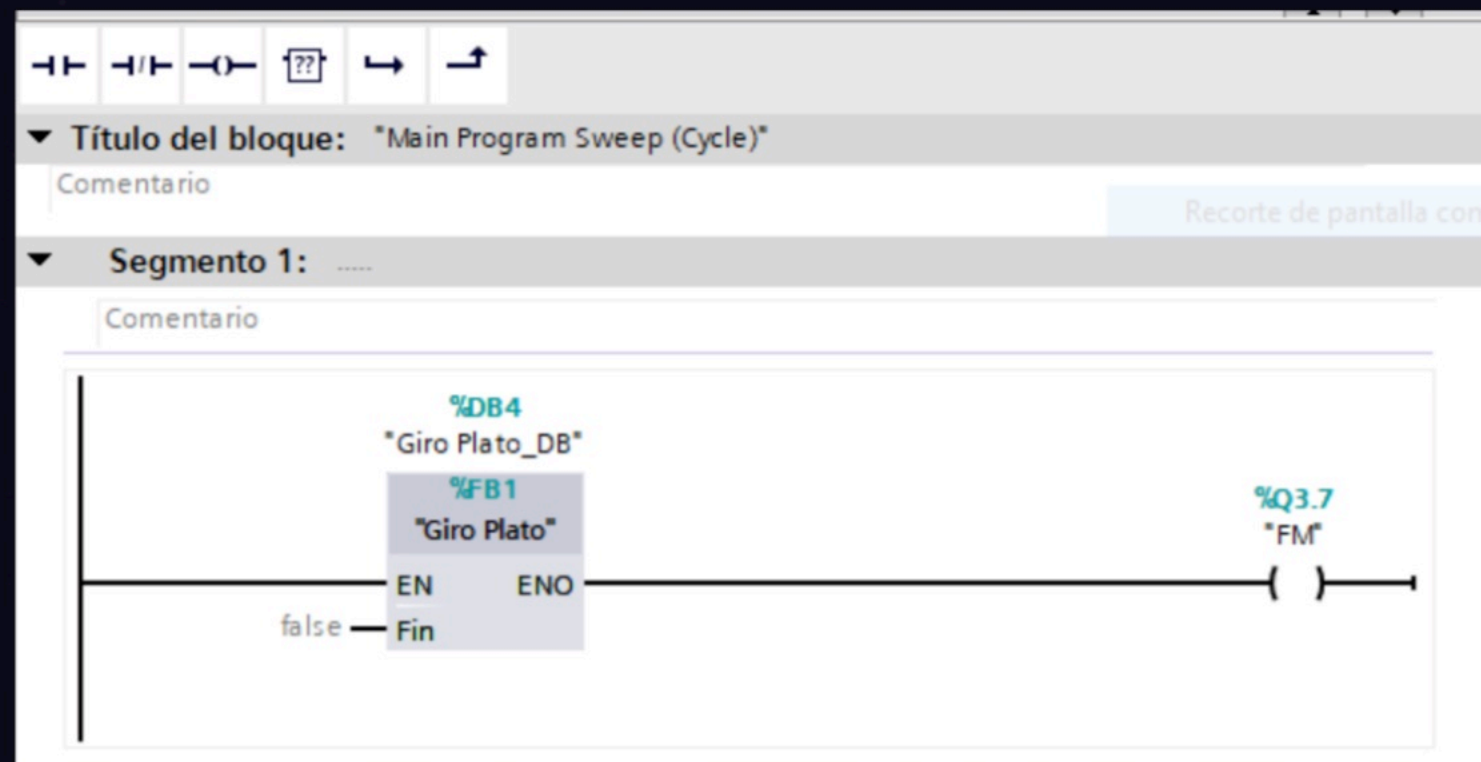
Implementacion de FC y FB en SCL

04

FC y FB



- No guardan datos de un ciclo SCAN a otro, por lo que no tienen asociados un DB
- Útil para cálculos matemáticos y tareas repetitivas*



FC

VARIABLES INTERNAS

Variables Internas

IN

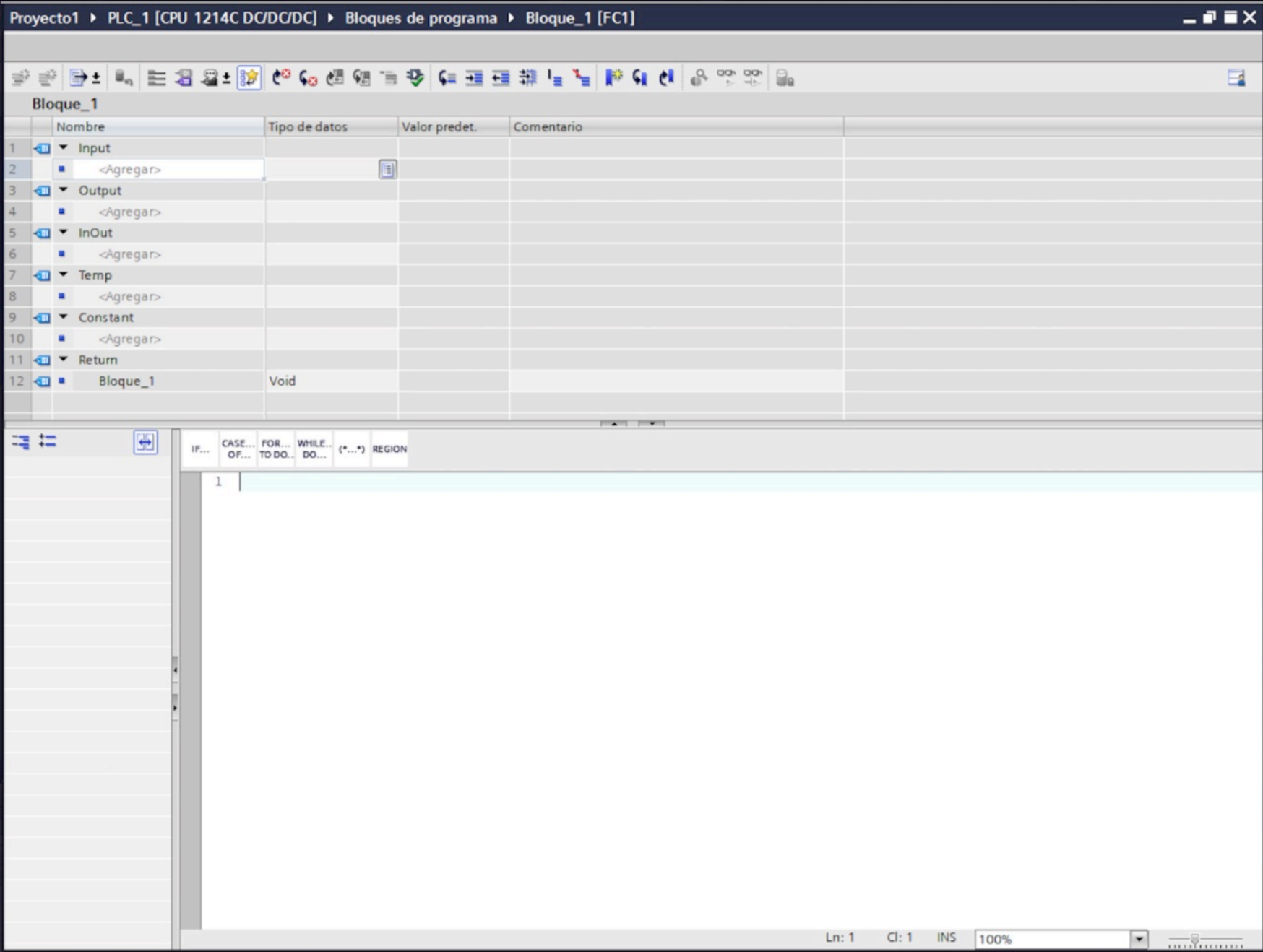
OUT

INOUT

TEMP

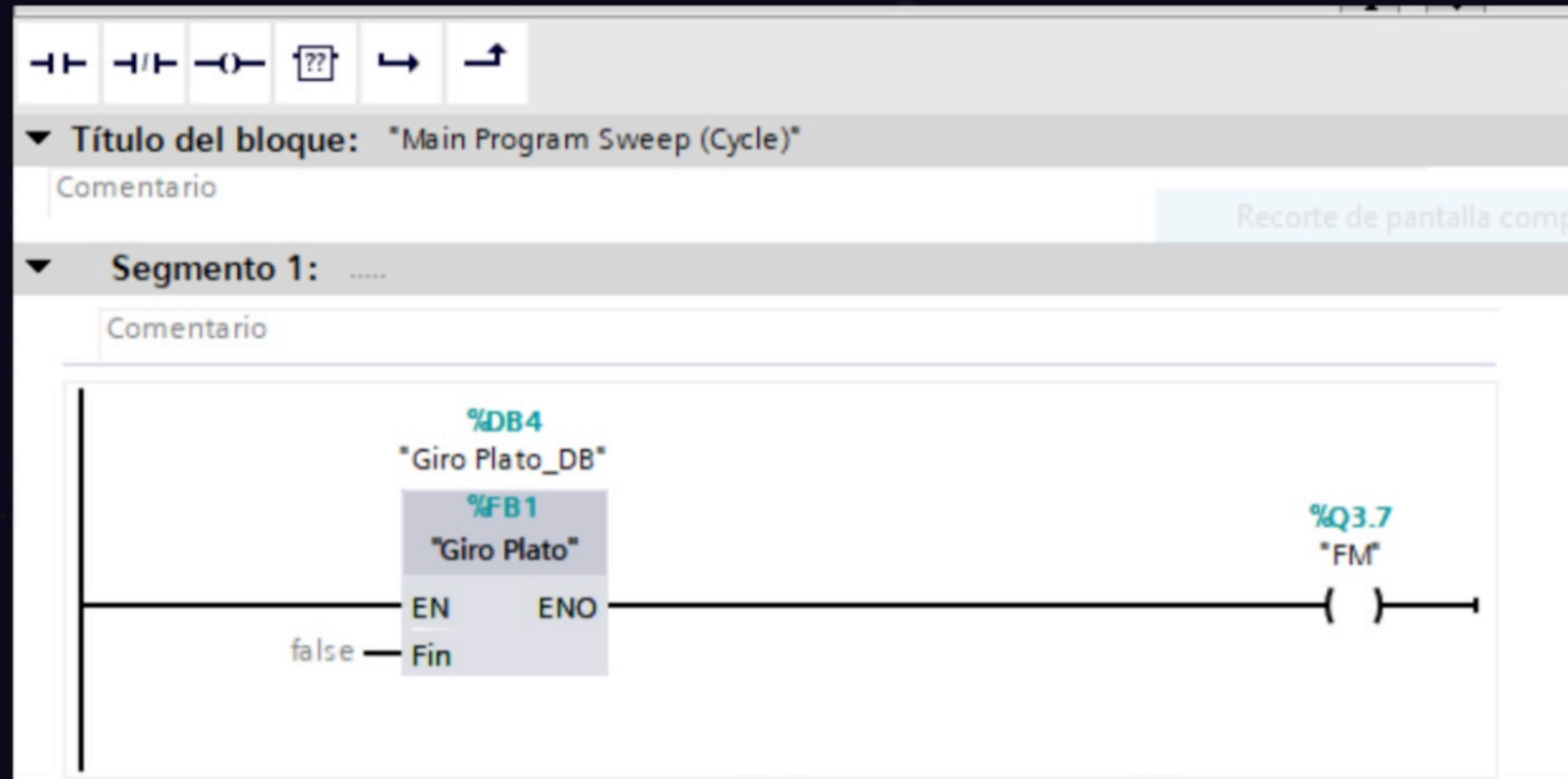
RETURN

FC y FB





- Guardan los datos de un ciclo SCAN a otro, por lo que están asociados a un DB
- Útil cuando es necesario saber la etapa en la que se encuentra el ciclo



VARIABLES INTERNAS

Variables Internas

IN

OUT

INOUT

STATIC

CONSTANT

Proyecto1 ▶ PLC_1 [CPU 1214C DC/DC/DC] ▶ Bloques de programa ▶ Bloque_1 [FB5]

Bloque_1

	Nombre	Tipo de datos	Valor predet.	Remanencia	Accesible d...	Escrib...	Visible en ..	Valor de a..	Comentario
1	▼ Input				☐	☐	☐	☐	
2	■ <Agregar>			▼	☐	☐	☐	☐	
3	▼ Output				☐	☐	☐	☐	
4	■ <Agregar>				☐	☐	☐	☐	
5	▼ InOut				☐	☐	☐	☐	
6	■ <Agregar>				☐	☐	☐	☐	
7	▼ Static				☐	☐	☐	☐	
8	■ <Agregar>				☐	☐	☐	☐	
9	▼ Temp				☐	☐	☐	☐	
10	■ <Agregar>				☐	☐	☐	☐	
11	▼ Constant				☐	☐	☐	☐	
12	■ <Agregar>				☐	☐	☐	☐	

IF... CASE... FOR... WHILE... (*...*) REGION OF... TO DO... DO...

1

TRANSCRIPCIÓN DE EJEMPLOS MÁS COMUNES A SCL

Detección de flanco

05



Se desea controlar una atracción de feria mediante un PLC (Fig. 1). La atracción se pondrá en movimiento tras insertar 4 fichas en el monedero. Por cada ficha introducida el monedero activará una salida durante un tiempo indeterminado, muy superior al ciclo SCAN del PLC.

Una vez introducidas el número de fichas se activará una señal luminosa, indicando que se puede pulsar el pulsador de marcha (PM) para el comienzo del viaje.

El viaje consiste en 20 movimientos de vaivén del balancín y posterior activación de una señal acústica durante 5s. Este ciclo se debe repetir 10 veces. El movimiento del balancín se realiza mediante un cilindro de doble efecto pilotado indirectamente por una válvula biestable. La posición del cilindro se detecta mediante finales de carrera.

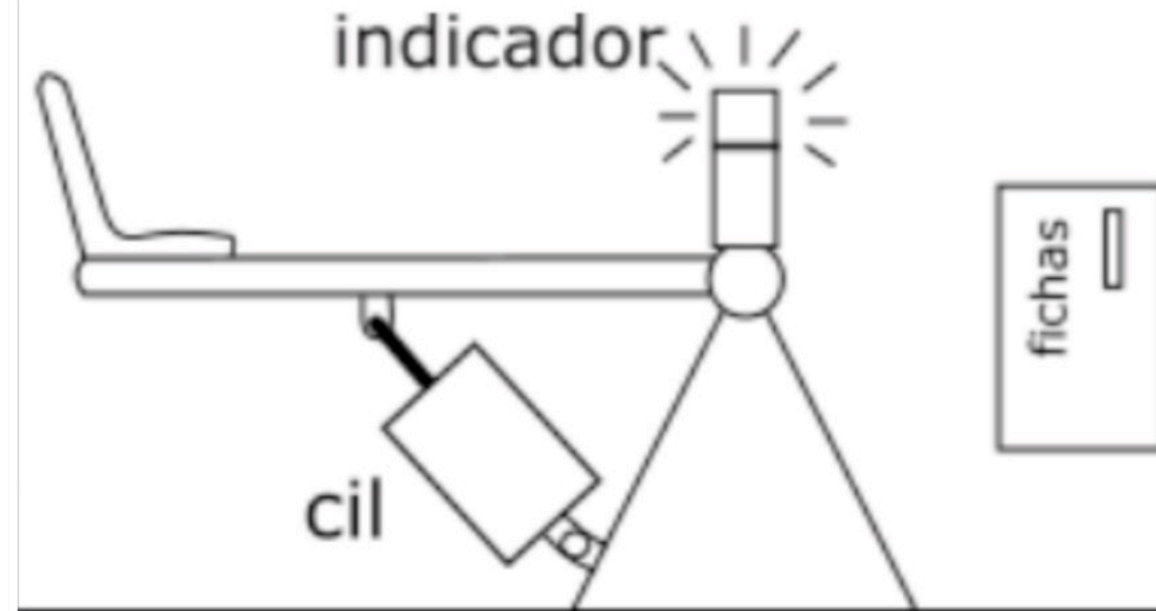


Figura 1 Atracción de feria

```

1 // Contar fichas del monedero (una por pulso)
2 //
3 IEC_Counter_0_DB_1.CTU(CU:=#Monedero,
4   R:="Rc",
5   PV:=4,
6   Q=>#Qc);
7
8
9 #Luz := #Qc; // Se enciende cuando hay 4 fichas
10
11 // Detectar flanco de subida
12 #PM_Flanco := (#PM = TRUE) AND (#PM_Anterior = FALSE);
13
14 // Actualizar el estado anterior del botón para el próximo ciclo
15 #PM_Anterior := #PM;
16 IEC_Timer_0_DB_9.TON(IN:=#IN1,
17   PT:=T#5s,
18   Q=>"Q1");
19
20

```

```

20
21 CASE "Estado" OF
22   0: // Espera 4 fichas
23     "Rc" :=false;
24     IF #Qc THEN
25       "Estado" := 1;
26     END_IF;
27   1: // Espera pulsador de marcha
28     IF #PM_Flanco THEN
29       "c" := 0;
30       "Estado" := 2;
31     END_IF;
32   2: // Avanzar cilindro
33     "A+":=TRUE;
34     IF "Ae" THEN
35       #IN1 := True;
36       "A+":= FALSE
37     ;
38     "Estado" := 3;
39   END_IF;
40   3: // Espera 5s con cilindro extendido
41     IF "Q1" THEN
42       #IN1 := FALSE;
43
44     ;
45     "Estado" := 4;
46   END_IF;
47   4: // Retraer cilindro
48     "A-":=TRUE;
49     IF "Ar" THEN
50       "A-":= FALSE;
51       "c" := "c" + 1;
52       IF "c" < 10 THEN
53         "Estado" := 2; // Repetimos ciclo
54       ELSE
55         "Estado" := 5; // Fin del proceso
56       END_IF;
57     END_IF;
58   5: // Ciclo completo, reset sistema
59     "Rc" :=true;
60     "Estado" := 0;
61 END_CASE;
62
63

```

CONCLUSIONES

VENTAJAS

01

Integración

Faciles de integrar tanto en bloques programados en SCL o KOP

02

Depuración

Ser un lenguaje estructurado proporciona mayor facilidad de depuración ante fallos de programación.

03

Tratamiento de datos

Permite trabajar con un rango mayor de tipos de variables, facilitando el procesamiento y tratamiento de datos

04

Eficiencia

Permite reducir las variables internas, lo que deriva en un programa más limpio y eficiente

CONCLUSIONES

CONCLUSIÓN FINAL

06

El lenguaje SCL presenta cierta complejidad a la hora de desarrollar la totalidad del programa exclusivamente con él. Sin embargo, se posiciona como una herramienta muy útil para estructurar y simplificar las etapas más complejas del proceso, permitiendo su integración posterior en KOP de forma más ordenada y eficiente.

BIBLIOGRAFÍA



- Mejía Arango, J. G. (s.f.). Programación de controladores lógicos programables con lenguaje SCL. [Material didáctico].
- Siemens. (s.f.). Documentación didáctica para cursos de formación: Programación en lenguajes de alto nivel con SCL y SIMATIC S7-1200 (Módulo TIA Portal 051-201). Siemens Automation Cooperates with Education (SCE).

¡GRACIAS!